MON APPRENTISSAGE — SERIES PYTHON

Python

Du zéro à l'expert

Chapitre 25 — Machine Learning avec Scikit-learn

fit, predict, score — apprendre à faire apprendre les machines

Auteur :

KETOTSA AMÉVI CLAUDE

Date de publication :

23 mars 2026

Publié dans le cadre de mon parcours d'apprentissage Python

Table des matières

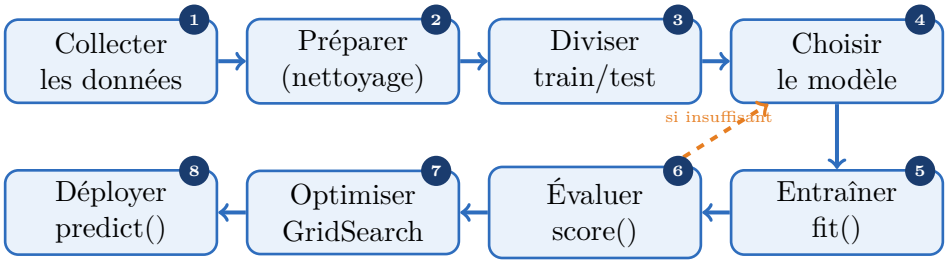
Chapitre 25 — Machine Learning avec Scikit-learn **2**
0.1 Le workflow du Machine Learning 2
0.2 Régression — Prédire des valeurs continues 2
0.3 Classification — Prédire une catégorie 4
0.4 Clustering — Regrouper sans étiquettes 5
0.5 Pipeline — Enchaîner les étapes 7
0.6 Ce qui m'a le plus surpris dans ce chapitre 8
0.7 Résumé du Chapitre 25 9

Chapitre 25 — Machine Learning avec Scikit-learn

À retenir

Le **Machine Learning** est la capacité pour une machine d'**apprendre** à partir de données, sans être explicitement programmée pour chaque cas. **Scikit-learn** est la bibliothèque ML de référence en Python — cohérente, bien documentée, et couvrant l'essentiel des algorithmes classiques. Ce chapitre couvre le workflow complet : préparation des données, entraînement, évaluation et optimisation des modèles.

Le workflow du Machine Learning



Régression — Prédire des valeurs continues

Régression linéaire — prédire un salaire

```python
import numpy as np
import pandas as pd
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import (mean_squared_error,
                             mean_absolute_error, r2_score)
from sklearn.preprocessing import StandardScaler

np.random.seed(42)
n = 300

# Generer des donnees synthetiques
df = pd.DataFrame({
    "experience":  np.random.uniform(0, 20, n),
    "formation":   np.random.choice([12,15,17,20], n),
    "age":         np.random.uniform(22, 55, n),
})
# Salaire = fonction des features + bruit
df["salaire"] = (
    2000 * df["experience"] +
    1500 * df["formation"]  +
```

```python
    500  * df["age"]        +
    np.random.normal(0, 5000, n) + 10000
)

# Features et cible
X = df[["experience","formation","age"]]
y = df["salaire"]

# Division train/test     80% / 20%
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Normalisation     optionnelle pour LR mais bonne pratique
scaler  = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test  = scaler.transform(X_test)   # m me scaler !

# Entrainer le modele
modele = LinearRegression()
modele.fit(X_train, y_train)          #     apprentissage

# Predire
y_pred = modele.predict(X_test)

# Evaluer
mse  = mean_squared_error(y_test, y_pred)
mae  = mean_absolute_error(y_test, y_pred)
r2   = r2_score(y_test, y_pred)
rmse = np.sqrt(mse)

print(f"R    : {r2:.4f}     (1.0 = parfait)")
print(f"RMSE : {rmse:,.0f}     (erreur quadratique)")
print(f"MAE  : {mae:,.0f}     (erreur absolue)")

# Coefficients
features = ["experience","formation","age"]
for feat, coef in zip(features, modele.coef_):
    print(f"  {feat:15} : {coef:+.1f}")
print(f"  Intercept       : {modele.intercept_:.1f}")
```

Classification — Prédire une catégorie

Classification — détecter les abandons scolaires

```python
import numpy as np
import pandas as pd
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import (classification_report,
                             confusion_matrix,
                             accuracy_score, roc_auc_score)
from sklearn.preprocessing import LabelEncoder

np.random.seed(42)
n = 500

# Donnees synth tiques     predire l'abandon scolaire
df = pd.DataFrame({
    "note_moy":    np.random.uniform(5, 20, n),
    "absences":    np.random.poisson(3, n),
    "distance_km": np.random.exponential(10, n),
    "revenu_fam":  np.random.choice([1,2,3,4], n),
    "genre":       np.random.choice(["M","F"], n),
})
# Cible : abandon = 1 si conditions defavorables
df["abandon"] = (
    (df["note_moy"]    < 10) |
    (df["absences"]    > 8)  |
    (df["distance_km"] > 25)
).astype(int)
# Ajouter du bruit
flip = np.random.rand(n) < 0.05
df["abandon"] = df["abandon"] ^ flip

# Encodage variable categorielle
df["genre_enc"] = (df["genre"] == "M").astype(int)

X = df[["note_moy","absences","distance_km",
        "revenu_fam","genre_enc"]]
y = df["abandon"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42,
    stratify=y   # conserver les proportions
)

# Random Forest
rf = RandomForestClassifier(
    n_estimators=100, max_depth=5,
```

```
    random_state=42, class_weight="balanced"
)
rf.fit(X_train, y_train)
y_pred = rf.predict(X_test)
y_prob = rf.predict_proba(X_test)[:, 1]

# Metriques
print(f"Accuracy  : {accuracy_score(y_test, y_pred):.4f}")
print(f"AUC-ROC   : {roc_auc_score(y_test, y_prob):.4f}")
print("\nRapport d taille :")
print(classification_report(y_test, y_pred,
      target_names=["Non-abandon","Abandon"]))

# Importance des variables
importances = pd.Series(
    rf.feature_importances_, index=X.columns
).sort_values(ascending=False)
print("\nImportance des variables :")
for feat, imp in importances.items():
    barre = "    " * int(imp * 40)
    print(f"  {feat:15} {barre} {imp:.3f}")
```

Clustering — Regrouper sans étiquettes

K-Means — segmentation non supervisée

```
import numpy as np
import pandas as pd
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import silhouette_score

np.random.seed(42)
n = 400

# Donnees etudiants     profils sans etiquettes
df = pd.DataFrame({
    "note_moy":  np.concatenate([
        np.random.normal(16, 1.5, n//4),   # groupe excellence
        np.random.normal(13, 1.5, n//4),   # groupe bon
        np.random.normal(10, 1.5, n//4),   # groupe moyen
        np.random.normal(7,  1.5, n//4),   # groupe difficile
    ]),
    "assiduite": np.concatenate([
        np.random.normal(95, 3, n//4),
        np.random.normal(80, 5, n//4),
        np.random.normal(70, 8, n//4),
        np.random.normal(55, 10, n//4),
```

```
    ]),
    "participation": np.concatenate([
        np.random.normal(9, 0.5, n//4),
        np.random.normal(7, 1,   n//4),
        np.random.normal(5, 1.5, n//4),
        np.random.normal(3, 1,   n//4),
    ]),
})

# Normalisation
scaler = StandardScaler()
X_scaled = scaler.fit_transform(df)

# Methode du coude     trouver le k optimal
inertias   = []
silhouettes = []
K_range = range(2, 9)

for k in K_range:
    km = KMeans(n_clusters=k, random_state=42, n_init=10)
    labels = km.fit_predict(X_scaled)
    inertias.append(km.inertia_)
    silhouettes.append(silhouette_score(X_scaled, labels))

print("k | Inertie    | Silhouette")
print("-" * 35)
for k, ine, sil in zip(K_range, inertias, silhouettes):
    print(f"{k} | {ine:10.1f} | {sil:.4f}")

# Appliquer avec k=4
km_final = KMeans(n_clusters=4, random_state=42, n_init=10)
df["cluster"] = km_final.fit_predict(X_scaled)

# Profil de chaque cluster
profil = df.groupby("cluster").agg({
    "note_moy":      "mean",
    "assiduite":     "mean",
    "participation": "mean",
    "cluster":       "count"
}).rename(columns={"cluster": "effectif"})

noms = {0: "Excellence", 1: "Bon niveau",
        2: "Moyen",      3: "En difficulte"}
profil.index = [noms.get(i, f"C{i}")
                for i in profil.index]
print("\nProfil des clusters :")
print(profil.round(1))
```

Pipeline — Enchaîner les étapes

Pipeline scikit-learn — workflow propre

```python
import numpy as np
import pandas as pd
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.ensemble import GradientBoostingClassifier
from sklearn.model_selection import (train_test_split,
                                     cross_val_score,
                                     GridSearchCV)
from sklearn.metrics import classification_report

np.random.seed(42)
n = 400

df = pd.DataFrame({
    "note":    np.random.uniform(5, 20, n),
    "abs":     np.random.poisson(3, n),
    "dist":    np.random.exponential(10, n),
    "genre":   np.random.choice(["M", "F"], n),
    "region":  np.random.choice(["Nord","Sud","Est"], n),
})
df["risque"] = ((df["note"] < 10) |
                (df["abs"]  > 8)).astype(int)

X = df.drop("risque", axis=1)
y = df["risque"]

# Colonnes num riques et categorielles
num_cols = ["note", "abs", "dist"]
cat_cols = ["genre", "region"]

# Preprocesseur combine
preprocesseur = ColumnTransformer([
    ("num", StandardScaler(),    num_cols),
    ("cat", OneHotEncoder(
        handle_unknown="ignore"), cat_cols),
])

# Pipeline complet
pipeline = Pipeline([
    ("prep",   preprocesseur),
    ("modele", GradientBoostingClassifier(
                    random_state=42)),
])

X_train, X_test, y_train, y_test = train_test_split(
```

```
    X, y, test_size=0.2, random_state=42
)

# Validation croisee      evaluation robuste
scores = cross_val_score(pipeline, X_train, y_train,
                         cv=5, scoring="roc_auc")
print(f"AUC-ROC CV : {scores.mean():.4f} "
      f"   {scores.std():.4f}")

# GridSearch      optimisation des hyperparametres
param_grid = {
    "modele__n_estimators":  [50, 100, 200],
    "modele__learning_rate": [0.05, 0.1, 0.2],
    "modele__max_depth":     [3, 4, 5],
}

grid_search = GridSearchCV(
    pipeline, param_grid,
    cv=3, scoring="roc_auc",
    n_jobs=-1, verbose=1
)
grid_search.fit(X_train, y_train)

print(f"\nMeilleurs parametres :")
for k, v in grid_search.best_params_.items():
    print(f"  {k:35} : {v}")
print(f"Meilleur AUC-ROC : {grid_search.best_score_:.4f}")

# Evaluation finale
meilleur = grid_search.best_estimator_
y_pred   = meilleur.predict(X_test)
print("\nRapport final :")
print(classification_report(y_test, y_pred,
      target_names=["Non-risque","A risque"]))
```

Ce qui m'a le plus surpris dans ce chapitre

Ce qui m'a résisté

Trois révélations de ce chapitre :

1. L'API unifiée de Scikit-learn — `fit`, `predict`, `score`. Tous les modèles Scikit-learn — régression linéaire, Random Forest, SVM, K-Means — partagent exactement la même interface. Changer de modèle ne nécessite que de remplacer une ligne. Cette cohérence n'est pas du luxe — c'est un design philosophique profond qui permet d'expérimenter rapidement, de comparer les modèles objectivement, et d'apprendre une fois la logique valable pour tout l'écosystème.

2. Le scaler doit être `fit` sur le train, `transform` sur le test. J'avais normalisé

toutes mes données **avant** de faire le split. C'est une fuite de données (*data leakage*) — le scaler voit les données de test lors de son entraînement, et les métriques deviennent optimistes et trompeuses. La règle absolue : `fit_transform(X_train)` puis `transform(X_test)`. Le Pipeline de Scikit-learn force cette bonne pratique structurellement.

3. La validation croisée — l'honnêteté statistique. Évaluer sur un seul split train/test donne un résultat qui dépend du hasard de la division. La validation croisée à k-folds répète l'évaluation sur k découpages différents et donne une estimation robuste avec son intervalle de confiance. Pour un statisticien, c'est la différence entre une mesure et une estimation — fondamentale.

Résumé du Chapitre 25

Ce que j'ai appris dans ce chapitre :

- API unifiée Scikit-learn : `fit()`, `predict()`, `score()`.
- **Régression** : prédire des valeurs — R^2, RMSE, MAE.
- **Classification** : prédire des classes — accuracy, AUC-ROC, F1.
- **Clustering K-Means** : segmenter sans étiquettes.
- `train_test_split(stratify=y)` — diviser proprement.
- `StandardScaler` : fit sur train, transform sur test uniquement.
- `Pipeline` + `ColumnTransformer` — workflow sans data leakage.
- `cross_val_score` — évaluation robuste à k-folds.
- `GridSearchCV` — optimisation automatique des hyperparamètres.

"Le Machine Learning ne remplace pas le statisticien. Il lui donne des outils pour traiter en secondes ce qui lui prendrait des années à la main."